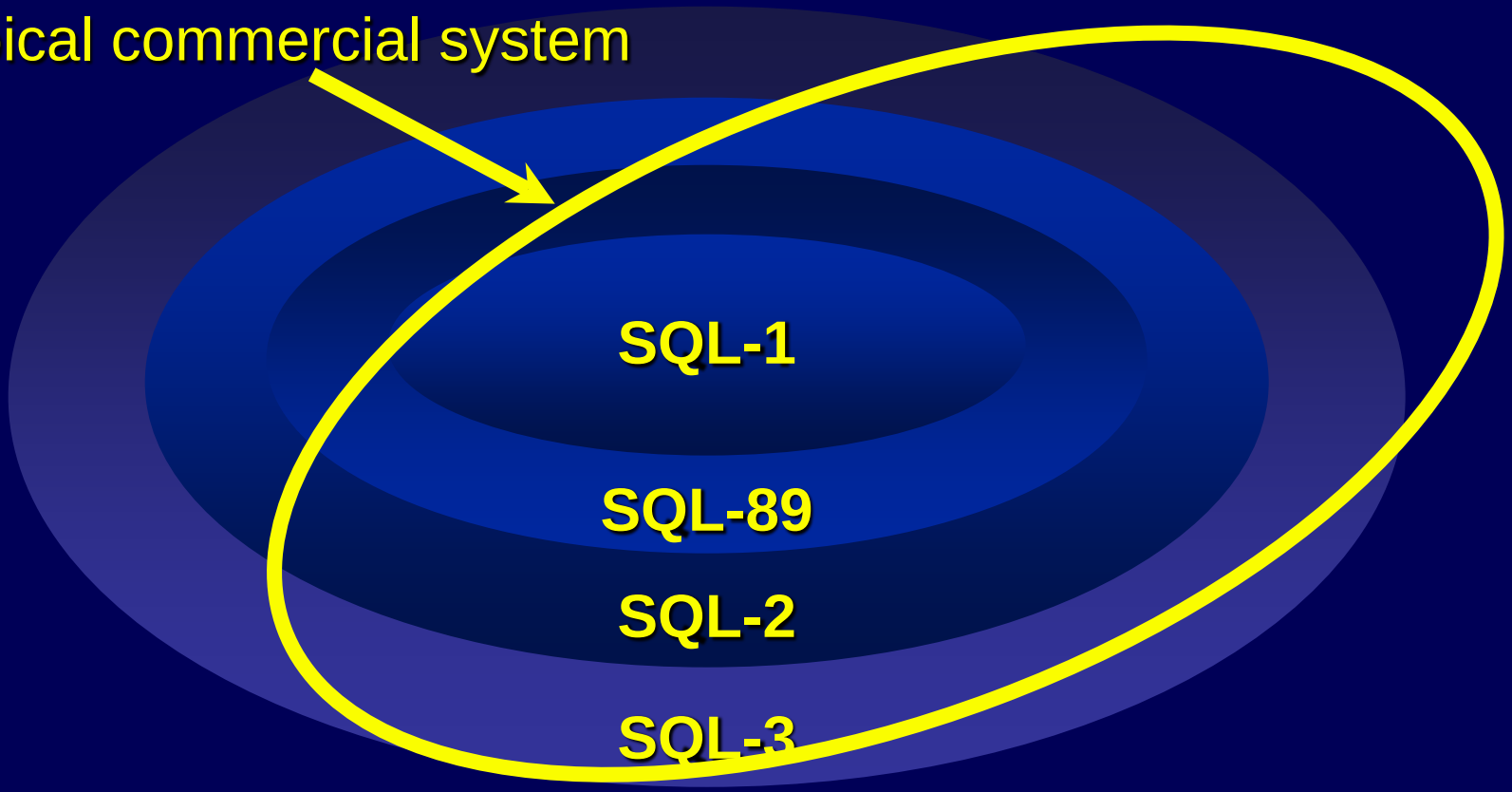
The SQL language
(part 1)
Intro, SPJ, set operators, sorting, aggregates

SQL

- The name stands for ***S**tructured **Q**uery **L**anguage*
  - DML: query, insert/delete/update, transactions
  - DDL: definitions for domains, tables, indexes, authorizations, views, constraints, procedures, triggers
- History:
  - First proposal: SEQUEL (IBM Research, 1974)
  - First commercial implementation: SQL/DS (IBM, 1981)
- Standardization was the main reason for its success (ANSI,ISO)
  - 1986 (SQL-1), 1989 (SQL-89), 1992 (**SQL-2** or **SQL-92**), 1999 (SQL-3 or SQL:1999)
  - SQL-92 has 3 levels of standardization: **Entry** SQL (close to SQL-89) / **Intermediate** / **Full**
- Most commercial systems comply to the **entry level** specs, and offer **proprietary extensions** for advanced functionalities

SQL Standards vs Commercial Systems

A typical commercial system



SQL – a Query Language

- **SQL is a declarative** query language:
  - Users state which information is required, not how to extract it
- The programmer focuses on the readability of the query, not on its execution efficiency
  - A query optimizer translates the queries into an internal procedural (“algebraic”!) language within the DBMS
- This is one of the most qualifying aspects of relational database systems

SQL Queries

SQL Queries

- Syntax: `select [ distinct ]` *AttrExpr* {`,` *AttrExpr*}
  `from` *Table* {`,` *Table*}
  `[ where` *Condition* `]`
- Three clauses:
  - `select` (also called the “target list”), `from`, `where` (optional)
- The query expresses a Cartesian product of the tables in the `from` clause, and returns those tuples selected by the `where` clause, projected to the attributes in the `select` clause
- The `where` clause is evaluated **tuple by tuple**
- Duplicate tuples are removed only if `distinct` is specified

Notation for syntax patterns

- **Syntax specification**
  - `from` → a keyword of the language
  - *AttrExpr* → “placeholder" to be replaced with a value or expression
  - [x] → x is optional (can be omitted)
  - { x } → x is repeated any number of times (including 0)
  - < x | y | z > → only one (x, y or z) can be chosen

Full Syntax for a simple query

```
select [ distinct ] AttrExpr [ [ as ] Alias ]
        { , AttrExpr [ [ as ] Alias ] }
from Table [ [ as ] Alias ]
     { , Table [ [ as ] Alias ] }
[ where Condition ]
```

Algebraic Interpretation of SQL

A generic query:

```
select Ti.Attribute_h, ..., Tj.Attribute_k
from Table_1 as T1, ..., Table_N as Tn
where Condition
```

BUT FOR DUPLICATES

- "corresponds" to the algebraic expression:

$$T1 := Table_1 \quad \ldots \quad Tn := Table_N$$

$$\pi_{Ti.Attribute_h,Tj.Attribute_k}\left(\sigma_{Condition}\left(T1 \times \ldots \times Tn\right)\right)$$

Example: Exams

Student

SId	Name	City	Dept
123	Carlo	Bologna	CS
415	Paola	Torino	CS
702	Antonio	Roma	Log

Exam

SId	CourseId	Date	Grade
123	1	7-9-13	30
123	2	8-1-14	28
702	2	7-9-13	25

Course

CourseId	Title	Professor
1	Math	Barozzi
2	Databases	Meo

A very simple query

```
select Name
from Student
where Dept = 'Log'
```

Name
Antonio

Algebraic interpretation (but for duplicates) :

$$\Pi_{\text{Name}}\ \sigma_{\text{Dept='Log'}}\ \textbf{Student}$$

Another Simple Query

```
select *
from Student
```

SId	Name	City	Dept
123	Carlo	Bologna	CS
415	Paola	Torino	CS
702	Antonio	Roma	Log

Syntax for the `select` clause

```
select *
select Name, City
select distinct City
select City as Residence
select (today()-Birthdate)*24 as AgeInHrs
select sum(Salary)
```

Duplicates

- Queries may return tables with identical rows
- Duplicates can be explicitly removed with `distinct`

```
select Dept
from Student
```

Dept
CS
CS
Log

```
select distinct Dept
from Student
```

Dept
CS
Log

Syntax for the `where` clause

- Boolean expression of simple predicates (just like in Relational Algebra)
- Some additional predicates:
  - **`between:`**

    ```
    Date between '1990-01-01' and '1999-12-31'
    ```

  - **`like:`**

    ```
    Dept like ’Lo%’
    PlateNumber like ’MI_777_8%'
    ```

Conjunction of predicates

- Find the students of CS from Bologna

```
select *
from Student
where Dept = "CS" and
      City = "Bologna"
```

- Result:

SId	Name	City	Dept
123	Carlo	Bologna	CS

Disjunction of predicates

- Find the students "*from Bologna* ***and*** *Torino*":

```
select *
from Student
where City = "Bologna" or City = "Torino"
```

- Result:

SId	Name	City	Dept
123	Carlo	Bologna	CS
415	Paola	Torino	CS

- Beware! The `where` clause filters the tuples **one at a time**…
  the following condition is a contradiction (→ empty result!):

```
where City = "Bologna" and City = "Torino"
```

Boolean Expressions

- Students from Rome studying “CS” **or** “Log”:

```
select *
from Student
where City = 'Roma' and
      (Dept = 'CS' or Dept = 'Log')
```

- Result:

SId	Name	City	Dept
702	Antonio	Roma	Log

- Here the parenthesis are necessary to force the precedence (otherwise *and* prevails over *or*)

The `like` operator

- Find the students whose name has an ‘a’ as second char and ends by ‘o’ :

```
select *
from Student
where Name like '_a%o'
```

- Result:

SId	Name	City	Dept
123	Carlo	Bologna	CS

`null` values

- Null values may represent different situations:
  - the value does not apply
  - the value applies but remains unknown
  - the value may/may not apply

- SQL-89 uses a two-value logic (2VL):
  - a comparison with *null* returns FALSE
- SQL-2 uses a three-value logic (3VL):
  - a comparison with *null* returns UNKNOWN

- To check if an attribute has null value:

  Attribute `is [ not ] null`

Queries over **`null`** values

```
select *
from Student
where City is [not] null
```

if `City` is *null* then the predicate `(City = 'Milano')` returns UNKNOWN

If the predicate evaluates to **UNKNOWN** then the tuple is **not extracted** (*just like what happens if it evaluates to* **FALSE**), but we must be aware that it is the case!...

Predicates and null values

- **3-value logic** (T,F,U)

  T and **U** = U
  F and U = F

  T or U = T
  F or **U** = U

  U and U = U
  U or U = U

  not U = U

A predicate P :
(City is not null) and
(Dept like 'Lo%')

City	Dept
Milano	Log
Milano	NULL
NULL	Log
Milano	CS

P	Tuple is extracted
T	yes
U	no
F	no
F	no

Queries over **`null`** values

```
select *
from Student
where Dept = 'CS' or
      Dept <> 'CS'
```

is equivalent to:

```
select *
from Student
where Dept is not null
```

```
select *
from Student
where Dept='CS' or
      Dept<>'CS' or
      Dept is null
```

contains a *tautology* in a 3VL, and is equivalent to:

```
select *
from Student
```

Set Queries

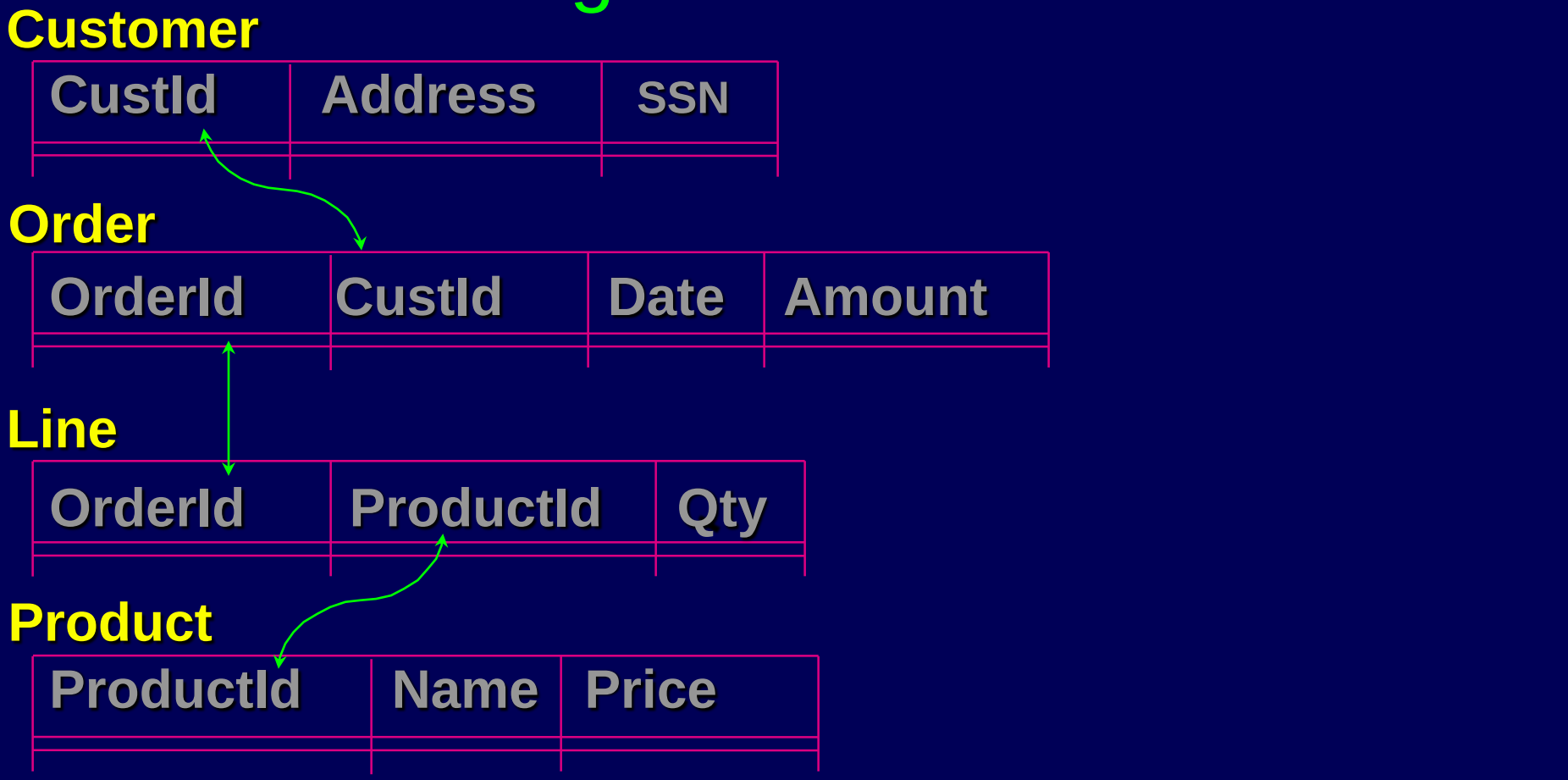
Set operators can be applied to two SQL queries. Syntax:

SelectSQL
< `union` | `intersect` | `except` > [`all`]
SelectSQL

- To respectively express union, intersection and difference between the results of the two queries

Duplicates are removed, unless `all` is specified

Management of Orders



Union

Find the id of the orders with an amount over 500 € or a product ordered with quantity over 1000

```
select OrderId
from Order
where Amount > 500
   union
select OrderId
from Line
where Qty > 1000
```

Positional Notation

Paternity

Father	Kid
Luigi	Giorgio
Stefano	Giovanni

Maternity

Mother	Kid
Anna	Giorgio
Paola	Giovanni

- The tables above represent maternity and paternity. What do the following queries extract?

```
select Father, Kid
from Paternity
  union
select Kid, Mother
from Maternity
```

```
select Father, Kid
from Paternity
  union
select Mother, Kid
from Maternity
```

Positional Notation

```
select Father, Kid
from Paternity
  union
select Kid, Mother
from Maternity
```



Luigi	Giorgio
Stefano	Giovanni
Giorgio	Anna
Giovanni	Paola

```
select Father, Kid
from Paternity
  union
select Mother, Kid
from Maternity
```



Luigi	Giorgio
Stefano	Giovanni
Anna	Giorgio
Paola	Giovanni

Attribute names in the result table

```
select Father
from Paternity
  union
select Mother
from Maternity
```

- Which are the column names of the result table?
  - No name?
  - Those of the first query?
  - ...?

The `all` keyword

- Find the fathers with kids named “Giorgio” or “Giovanni”, counting twice those fathers with a kid named Giorgio and a kid named Giovanni

```
select Father
from Paternity
where Kid = 'Giorgio'
   union all
select Father
from Paternity
where Kid = 'Giovanni'
```

Difference

- Find the OrderId of the orders with a total amount over 500€, but no product with a **quantity** exceeding **1000**

```
select OrderId
from Order
where Amount > 500
   except
select OrderId
from Line
where Qty > 1000
```

Difference

- Find the orders that have
  (i) *n* lines with a quantity of 10 at most (with $n > 0$), and
  (ii) *m* lines of order with a quantity exceeding 10, with ***n > m***

```
select OrderId
from Line
where Qty <= 10      n lines
   except all
select OrderId
from Line
where Qty > 10       m lines
```

Intersection

- Find orders with amount over 500€ **and** at least one product with quantity over 1000

```
select OrderId
from Order
where Amount > 500
   intersect
select OrderId
from Line
where Qty > 1000
```

Syntax for the `from` clause

```
from Student

from Student as X

from Student, Exam

from Student join Exam
     on Student.SId = Exam.SId
```

A 2-Table Query

Find the names of the students from the Logistics Dept with a 30

```
select distinct Name
from Student, Exam
where Student.SId = Exam.SId
      and Dept like 'Log%' and Grade = 30
```

$\Pi_{\text{Name}}\ \sigma_{\text{Dept} \approx \text{'Log\%'} \wedge \text{Grade=30} \wedge \text{Student.SId=Exam.SId}}$ (Student × Exam)

Π_{Name} ($\sigma_{\text{Dept} \approx \text{'Log\%'}}$ Student ⋈ $\sigma_{\text{Grade=30}}$ Exam)

Name
Antonio

A 3-Table Query

Find the names of the students with a 30 in “Math”

```
select distinct Name
from Student, Exam, Course
where Student.SId = Exam.SId
   and Exam.CourseId = Course.CourseId
   and Title like 'Mat%' and Grade = 30
```

$$\Pi_{\text{Name}}\ \sigma_{\text{Title} \approx \text{'Mat\%'} \wedge \text{Grade}=30 \wedge \text{Student.SId}=\text{Exam.SId} \wedge \text{Exam.CourseId}=\text{Course.CourseId}}\ (\text{Student} \times \text{Exam} \times \text{Course})$$

$$\Pi_{\text{Name}}\ \sigma_{\text{Title} \approx \text{'Mat\%'} \wedge \text{Grade}=30}\ (\text{Student} \bowtie \text{Exam} \bowtie \text{Course})$$

$$\Pi_{\text{Name}}\ (\text{Student} \bowtie \sigma_{\text{Grade}=30}\ \text{Exam} \bowtie \sigma_{\text{Title} \approx \text{'Mat\%'}}\ \text{Course})$$

Joins in the `from` clause

- Joins can be expressed within the `from` clause:

  `select` *AttrExpr* [[`as`] *Alias*] {, *AttrExpr* [[`as`] *Alias*] }
  `from` *Table* [[`as`] *Alias*]
  { [*JoinType*] `join` *Table* [[`as`] *Alias*] [`on` *Condition*] }
  [`where` *OtherConditions*]

- *JoinType* can be
  - `inner`
  - `natural`
  - < `right` | `left` | `full` > [`outer`]

Joins in the `from` clause

```
select Name
from Student, Exam
where Student.SId = Exam.SId
      and Dept like ’Log%’ and Grade = 30

select Name
from Student inner join Exam on Student.SId = Exam.SId
where Dept like ’Log%’and Grade = 30

select Name
from Student S inner join Exam E on S.SId = E.SId
where Dept like ’Log%’and Grade = 30

select Name
from Student natural join Exam
where Dept like ’Log%’and Grade = 30
```

Outer joins: Cars and Drivers

DRIVER

FirstName	Surname	DriverID
Mary	Brown	VR 2030020Y
Charles	White	PZ 1012436B
Marco	Neri	AP 4544442R

AUTOMOBILE

CarRegNo	Make	Model	DriverID
ABC 123	BMW	323	VR 2030020Y
DEF 456	BMW	Z3	VR 2030020Y
GHI 789	Lancia	Delta	PZ 1012436B
BBB 421	BMW	316	MI 2020030U

- There is a car with no driver (the BMW 316)
- There is a driver with no car (Marco Neri)

Inner join (aka… a **join**!)

- Find the drivers with their respective cars:

```
select FirstName, Surname, D.DriverID,
       CarRegNo, Make, Model
from Driver D inner join Automobile A on
     D.DriverID = A.DriverID
```

- Result:

FirstName	Surname	D.DriverID	CarRegNo	Make	Model
Mary	Brown	VR 2030020Y	ABC 123	BMW	323
Mary	Brown	VR 2030020Y	DEF 456	BMW	Z3
Charles	White	PZ 1012436B	GHI 789	Lancia	Delta

Natural join (*inner*, of course!)

- Find the drivers with their respective cars:

```
select FirstName, Surname, DriverID,
       CarRegNo, Make, Model
from Driver inner natural join Automobile
```

- Result:

FirstName	Surname	DriverID	CarRegNo	Make	Model
Mary	Brown	VR 2030020Y	ABC 123	BMW	323
Mary	Brown	VR 2030020Y	DEF 456	BMW	Z3
Charles	White	PZ 1012436B	GHI 789	Lancia	Delta

Left join (outer!)

- Find the drivers with their respective cars, also including drivers with no car:

```
select FirstName, Surname, D.DriverID,
       CarRegNo, Make, Model
from Driver D left outer join Automobile A
       on D.DriverID = A.DriverID
```

- Result:

FirstName	Surname	D.DriverID	CarRegNo	Make	Model
Mary	Brown	VR 2030020Y	ABC 123	BMW	323
Mary	Brown	VR 2030020Y	DEF 456	BMW	Z3
Charles	White	PZ 1012436B	GHI 789	Lancia	Delta
Marco	Neri	AP 4544442R	NULL	NULL	NULL

Right join (outer!)

- Find the drivers with their respective cars, also including cars with no driver:

```
select FirstName, Surname, A.DriverID,
       CarRegNo, Make, Model
from Driver D right outer join Automobile A
       on D.DriverID = A.DriverID
```

- Result:

FirstName	Surname	A.DriverID	CarRegNo	Make	Model
Mary	Brown	VR 2030020Y	ABC 123	BMW	323
Mary	Brown	VR 2030020Y	DEF 456	BMW	Z3
Charles	White	PZ 1012436B	GHI 789	Lancia	Delta
NULL	NULL	MI 2020030U	BBB 421	BMW	316

Full join (outer!)

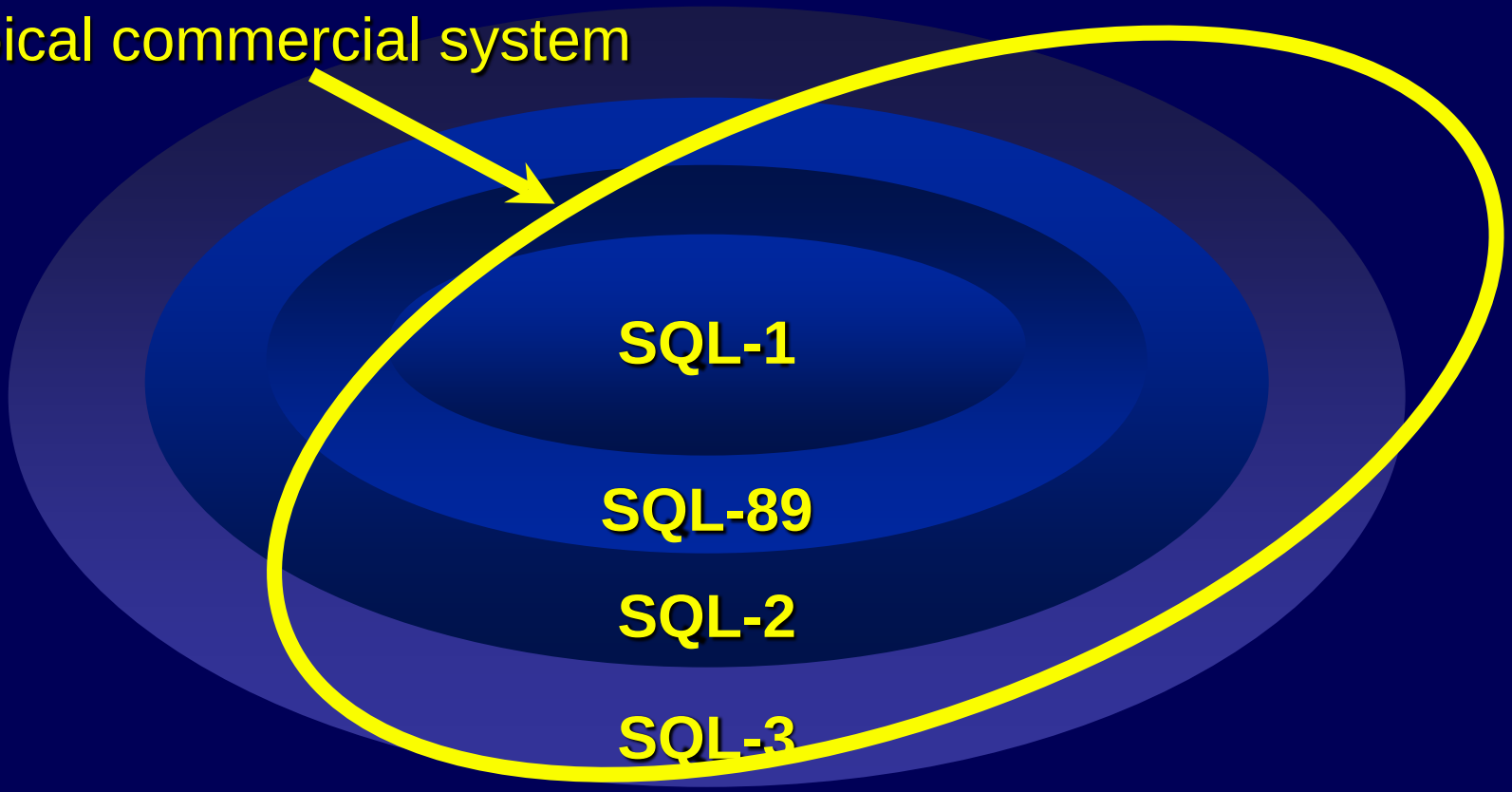
- Find the drivers with their respective cars, also including all unpaired tuples:

```
select FirstName, Surname, D.DriverID,
       A.DriverID, CarRegNo, Make, Model
from Driver D full outer join Automobile A
     on D.DriverID = A.DriverID
```

FirstName	Surname	D.DriverID	A.DriverID	CarRegNo	Make	Model
Mary	Brown	VR 2030020Y	VR 2030020Y	ABC 123	BMW	323
Mary	Brown	VR 2030020Y	VR 2030020Y	DEF 456	BMW	Z3
Charles	White	PZ 1012436B	PZ 1012436B	GHI 789	Lancia	Delta
Marco	Neri	AP 4544442R	NULL	NULL	NULL	NULL
NULL	NULL	NULL	MI 2020030U	BBB 421	BMW	316

SQL Standards vs Commercial Systems

A typical commercial system



Equivalence of expressions

- It is possible that a specific construct is not supported by a specific implementation
  - Example: **MySQL** does not support FULL outer joins (as of v. 8.0) !
- However, queries can often be rewritten into equivalent formulations using other constructs

```
select * from A full join B on A.x = B.y
```

can be easily rewritten as

```
select * from A left join B on A.x = B.y
  union
select * from A right join B on A.x = B.y
```

Variables in SQL

- The alias names of tables help in formulating queries more compactly
  - So as to improve readability
  - Using aliases is similar to the ρ operator (renaming) of Relational Algebra
- Aliases *must* be used whenever we need to refer to a table ***more*** *than once*
  - Otherwise, some expressions would be ambiguous

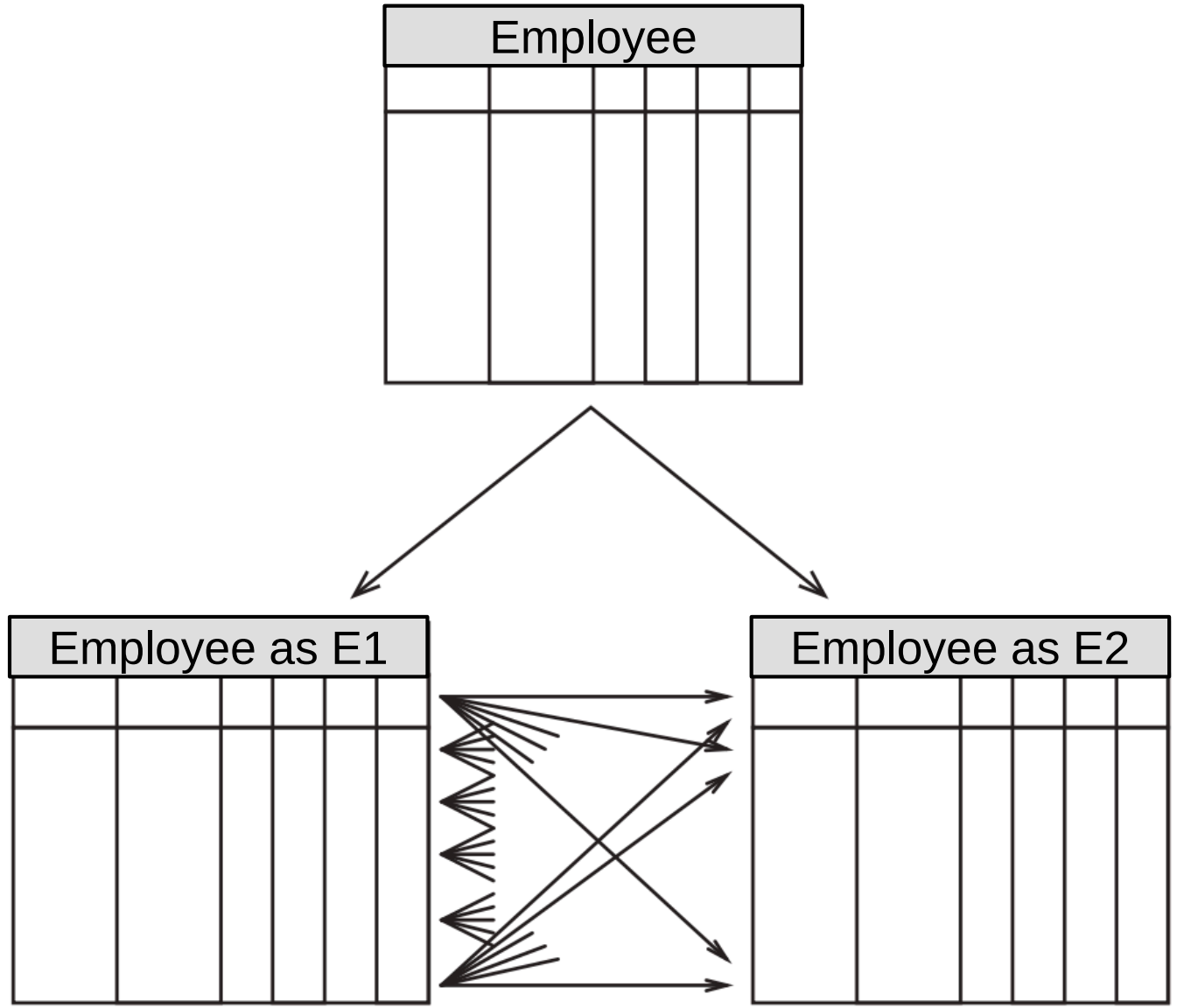
A Query with a “self-join”

Find the employees who report to «Giorgio»

Employee

Id	Name	HireDate	Salary	Manager
1	Piero	1-1-05	3 K	2
2	Giorgio	1-1-07	2,5 K	null
3	Marco	1-7-06	2 K	2

Self join of Employee



Employees with “Giorgio” as boss

```
select Emp.Name, Emp.Id, Bss.Id, Bss.Name
from Employee as Emp join Employee as Bss
        on Emp.Manager = Bss.Id
where Bss.Name = ’Giorgio’
```

Emp.Name	Emp.Id	Bss.Id	Bss.Name
Piero	1	2	Giorgio
Marco	3	2	Giorgio

So far, we just changed language, but we are still using «our algebra»

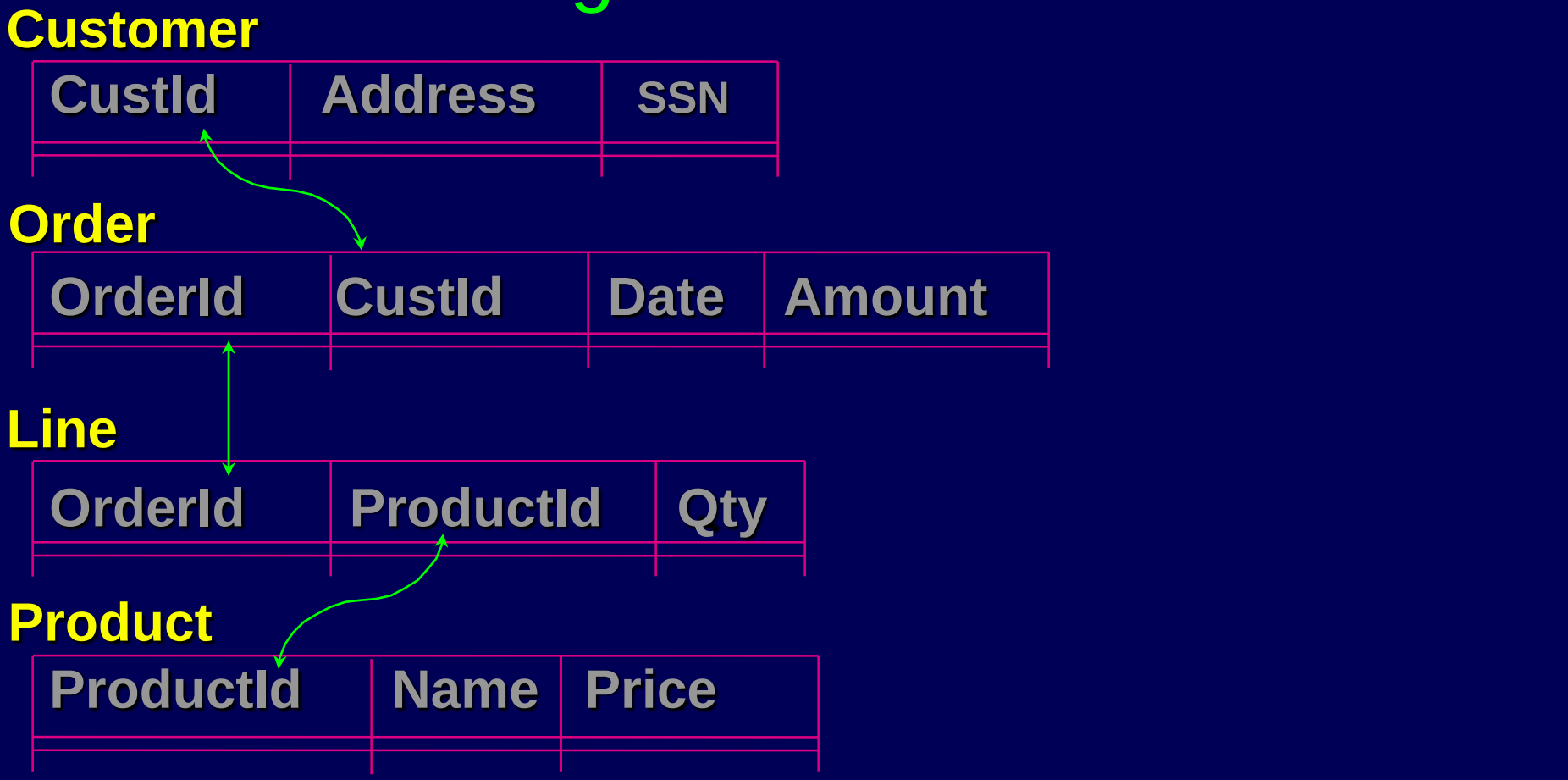
- But for the possibility to keep duplicates…
- …and the outer joins (that exist, b.t.w., in RA ⟕ ⟖ ⟗)

But SQL has a little more…

Complex Queries in SQL

- Sorting
  - `order by` *Attrib* [`asc` | `desc`]
- Aggregating
  - `count`, `sum`, `max`, `min`, `avg`
- Grouping
  - `group by` ... `having` ...
- Nesting
  - `where` *Attrib comp* (`select` ...)

Management of Orders



An Instance of table Order

Order

OrderId	CustId	Date	Amount
1	3	1-6-17	50.000,00
2	4	3-8-17	8.000,00
3	3	1-9-17	5.500,00
4	1	1-7-17	12.000,00
5	1	1-8-17	1.500,00
6	3	3-9-17	27.000,00

Sorting

- The `order by` clause sorts the returned tuples
- Syntax:

  `order by` *Attribute* [`asc` | `desc`]
  {, *Attribute* [`asc` | `desc`] }

- Sorting conditions are evaluated sequentially:
  - If two tuples show the same value for the first sorting attribute, the second sorting attribute is considered

Query - Sorting

```
select *
from Order
where Amount > 1000
order by Date
```

OrderId	CustId	Date	Amount
1	3	1-6-17	50.000,00
4	1	1-7-17	12.000,00
5	1	1-8-17	1.500,00
2	4	3-8-17	8.000,00
3	3	1-9-17	1.500,00
6	3	3-9-17	5.500,00

Order by CustId

orderId	CustId	Date	Amount
4	1	1-7-17	12.000,00
5	1	1-8-17	1.500,00
1	3	1-6-17	50.000,00
6	3	3-9-17	5.500,00
3	3	1-9-17	1.500,00
2	4	3-8-17	27.000,00

order by CustId asc, Date desc

OrderId	CustId	Date	Amount
5	1	1-8-17	1.500,00
4	1	1-7-17	12.000,00
6	3	3-9-17	5.500,00
3	3	1-9-17	1.500,00
1	3	1-6-17	50.000,00
2	4	3-8-17	27.000,00

Aggregation

- The results of the query depend on the aggregation of the considered tuples into sets (groups)
- The result of a query with aggregates depends on the evaluation of the contents of a set of rows
- SQL-2 provides the following operators:
  - `count` cardinality
  - `sum` sum
  - `max` maximum
  - `min` minimum
  - `avg` average

sum, max, min, avg

- Syntax:
  < sum | max | min | avg > ([distinct | all] *AttrExpr*)

- Option distinct considers each distinct value only once in evaluating the aggregate:
  - relevant for sum and avg, only
    - The other aggregates are not affected

- Option all considers all the values in the evaluation
  - Actually, only the values that are **not null**
  - all is the default option, if not specified

count

Syntax: `count( < * | [ distinct | all ] AttributeList > )`

- returns the number of…
  - …tuples, disregarding their content, if `*` is specified
  - …tuples with non-null values for the listed attributes, if `all` is specified (*this is the default option, and* `all` *can be omitted*)
  - …distinct non-null values for the listed attributes, if `distinct` is specified
- The number of orders:

```
select count(*) from Order
```

- The number of orders with a non-null value for attribute Amount :

```
select count(all Amount) from Order
```

- The number of distinct customers who placed at least one order:

```
select count(distinct CustId) from Order
```

Query with max

- Find the amount of the «heaviest» order:

```
select max(Amount) as MaxAmt
from Order
```

MaxAmt
50.000,00

Query with sum

- Find the sum of the amounts of the orders by customer 1

```
select sum(Amount) as SumAmt
from Order
where CustId = 1
```

SumAmt
13.500,00

Aggregate Functions with join

- Find the largest amount among orders that include the product with id ‘ABC’ :

```
select max(Amount) as MaxAmtABC
from Order natural join Line
where ProductId = ’ABC’
```

Aggregate Functions and Target List

- **A wrong** query:

```
select max(Amount), Date
from Order natural join Line
where ProductId = 'ABC'
```

- Which Date (of which order?) is to be extracted?

The target list must be homogeneous

- Find the largest and smallest amounts of the orders:

```
select max(Amount) as MaxAm,
       min(Amount) as MinAm
from Order
```

MaxAm	MinAm
50.000,00	1.500,00